Eliminating Serialization Leakage and Recovering Fine-Grained Multiclass Discrimination in IIoT Intrusion Detection via Privacy-Preserving Protocol-Header Structural Feature Engineering

Hamdan Ali

Department of Creative Technologies (Software Engineering), Faculty of Computing and Artificial Intelligence
Air University, Islamabad, Pakistan

Roll No.: 233120 | Section: F-23-BSSE-B | Email: Hamdanaly651@gmail.com

***Abstract*—**Machine learning-based Intrusion Detection Systems (IDS) evaluated on public Industrial Internet of Things (IIoT) benchmarks frequently report near-perfect classification accuracies (>99.8%). However, a forensic packet- and file-level audit of the widely adopted **Edge-IIoTset** benchmark conducted in this study reveals that such figures are substantially inflated by three previously undocumented or under-addressed dataset artifacts: (i) **mixed-type CSV serialization leakage**, where numeric zero values across five HTTP and MQTT protocol fields (`http.request.method`, `http.referer`, `http.request.version`, `mqtt.conack.flags`, and `mqtt.protoname`) are unevenly serialized as string `"0"` versus floating-point `"0.0"` across attack capture files due to multi-chunk `pandas` concatenation; (ii) **TShark delimiter-shift corruption**, where unescaped commas inside multicast DNS query strings shift string literals into numeric length columns (`dns.qry.name.len`); and (iii) **testbed topology memorization** via static IP addresses, capture timestamps, and raw hexadecimal payloads. Conversely, when recent studies strip all 15 identifier, port, and raw payload columns to enforce network-agnostic generalization, 15-class classification accuracy collapses to 84%–88% due to severe confusion among application-layer web attacks (`SQL_injection`, `XSS`, `Uploading`, `Password`) and transport-layer `DDoS_HTTP`. To resolve this accuracy–generalization trade-off without reintroducing identity leakage, we propose **Privacy-Preserving Protocol-Header Structural Feature Engineering (PP-PSFE)**, a deterministic transformation framework that extracts 24 non-identifying structural, geometric, and information-theoretic features—including canonical service/ephemeral port disentanglement, TCP option byte-length and timestamp indicators, Shannon byte entropy of raw TCP payloads, and application-layer URI query/payload boundary metrics—prior to discarding raw identifiers. Combined with 38 sanitized core protocol fields ($d = 62$ total features across $N = 152{,}386$ deduplicated flows), we conduct a strictly leakage-free stratified benchmark of four distinct learning paradigms: Multinomial Logistic Regression, Random Forest, LightGBM, and Regularized Second-Order XGBoost. Experimental results demonstrate that while the linear baseline plateaus at 84.87% test accuracy (0.8419 Macro-F1), **PP-PSFE + XGBoost** achieves **96.49% test accuracy**, **0.9618 Macro-F1**, **0.9991 weighted ROC-AUC**, and **0.9874 Macro PR-AUC** with a minimal +0.0057 generalization gap and 3.54 ms batch inference latency per 1,000 flows. Exact game-theoretic **TreeSHAP** attribution confirms that our engineered PP-PSFE features contribute **57.1% of total marginal log-odds magnitude** (73.88% of total split-gain importance). Finally, to bridge academic evaluation and industrial deployment, the fitted preprocessing-and-inference pipeline is packaged into a zero-leakage full-stack microservice architecture (**Flask REST API + Angular 22 standalone diagnostic interface + automated GitHub Actions CI/CD**).

***Index Terms*—**Industrial Internet of Things (IIoT), Intrusion Detection System (IDS), Edge-IIoTset, Data Leakage Sanitization, Protocol-Header Feature Engineering, Gradient Boosted Decision Trees, XGBoost, Explainable AI (TreeSHAP), Full-Stack MLOps.

I. INTRODUCTION

The rapid convergence of Operational Technology (OT) and Information Technology (IT) within smart manufacturing, automated power grids, and industrial telemetry networks has exposed formerly air-gapped Programmable Logic Controllers (PLCs), Message Queuing Telemetry Transport (MQTT) brokers, and edge gateways to sophisticated multi-vector cyberattacks [1], [2]. Unlike conventional enterprise networks where traffic is dominated by uniform web browsing, Industrial IoT (IIoT) edge segments carry heterogeneous, latency-sensitive machine-to-machine (M2M) flows spanning lightweight publish-subscribe telemetry (MQTT over TCP port 1883), Modbus/TCP register polling, address resolution (ARP), multicast service discovery (mDNS), and supervisory HTTP APIs [3]. A single compromised edge sensor can serve as a pivot point for volumetric transport-layer denial-of-service (`DDoS_TCP`, `DDoS_UDP`, `DDoS_ICMP`), reconnaissance (`Port_Scanning`, `Fingerprinting`, `Vulnerability_scanner`), credential brute-forcing (`Password`), web application exploitation (`SQL_injection`, `XSS`, `Uploading`), or remote command-and-control (`Backdoor`, `Ransomware`) [4].

To evaluate data-driven Intrusion Detection Systems (IDS) under realistic multi-protocol conditions, Ferrag *et al.* [4] introduced **Edge-IIoTset**, a comprehensive seven-layer testbed dataset encompassing more than 10 IoT sensor types and 14 distinct cyberattack categories alongside benign telemetry. Since its release, Edge-IIoTset has become a standard benchmark for

evaluating tree ensembles, deep neural networks (DNNs), convolutional-recurrent architectures (CNN-LSTM), and transformer-based traffic classifiers [5]–[10].

A. The Illusion of Near-Perfect Benchmark Accuracy

Despite the proliferation of studies reporting 99.5%–99.99% 15-class accuracy on Edge-IIoTset, industrial security practitioners repeatedly observe that models trained on public packet-capture (PCAP) benchmarks fail catastrophically when deployed on live edge gateways whose IP subnetting, capture window, or ephemeral port allocation differs from the laboratory testbed [11], [12]. Rather than accepting published benchmark numbers at face value, we initiated this research with a forensic byte- and column-level inspection of the raw Edge-IIoTset CSV distributions (`ML-EdgeIIoT-dataset.csv`, `DNN-EdgeIIoT-dataset.csv`, and the 24 per-attack and per-sensor capture files). Our audit uncovered three severe structural pathologies:

1. **Mixed-Type CSV Serialization Leakage (`"0"` vs. `"0.0"`):** In `ML-EdgeIIoT-dataset.csv` (157,800 rows, 63 columns), `pandas.read_csv` emits a `DtypeWarning` across nine columns. Inspecting individual class slices revealed that when the dataset creators concatenated separate PCAP-derived CSV chunks, missing protocol fields padded with numeric zero were exported as floating-point `"0.0"` in predominantly numeric chunks (e.g., 100% of `DDoS_UDP` and `DDoS_ICMP` records, and 99.9% of `Normal` records in `http.request.method`), but exported as integer string `"0"` in chunks containing string values (e.g., 93.5% of `DDoS_HTTP` and 96.8% of `XSS` records). When standard `LabelEncoder` or `OneHotEncoder` is applied directly to `df[col].astype(str)`, `"0"` and `"0.0"` receive two distinct category indices. Any tree ensemble or neural network immediately exploits this file-export artifact to separate attack classes without learning genuine network behavior.
2. **TShark CSV Delimiter-Shift Corruption:** In exactly 93 records (within `Normal` and `MITM` captures), unescaped commas inside multicast DNS query strings (`"_googlecast._tcp.local,..."`) caused TShark's CSV exporter to shift subsequent fields one column to the right, injecting string literals (`"_googlecast._tcp.local"` and `"null"`) directly into the numeric length column `dns.qry.name.len`.
3. **The Sanitization–Accuracy Collapse Dilemma:** When static testbed identifiers (`ip.src_host`, `ip.dst_host`, `frame.time`), raw ports (`tcp.srcport`, `tcp.dstport`), and raw hexadecimal/URI strings (`tcp.payload`, `tcp.options`, `http.file_data`, `http.request.full_uri`) are dropped alongside 8 zero-variance columns, the remaining 38 core protocol headers lose the structural discriminators required to separate application-layer attacks, causing linear 15-class accuracy to drop to 84.87%.

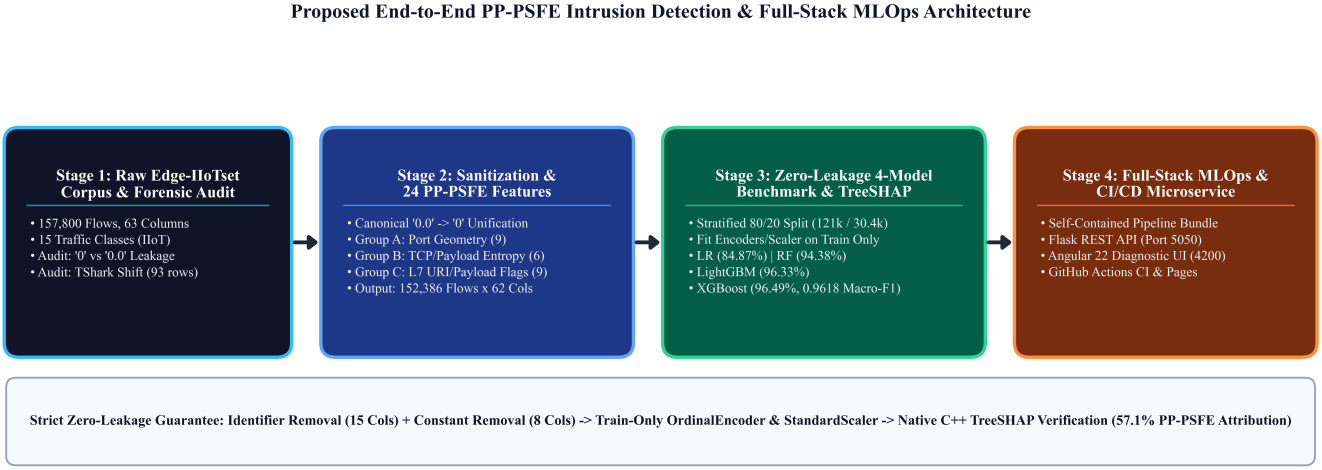


Fig. 1. Proposed four-stage system architecture: (Stage 1) Forensic audit of the 157,800-flow Edge-IIoTset corpus; (Stage 2) Canonical leakage sanitization and extraction of 24 Privacy-Preserving Protocol-Header Structural Features (PP-PSFE); (Stage 3) Zero-leakage stratified 4-model benchmark and native C++ TreeSHAP verification; and (Stage 4) Full-stack production MLOps deployment via Flask REST microservice, Angular 22 standalone dashboard, and GitHub Actions CI/CD.

B. Summary of Research Contributions

To resolve this fundamental tension between zero-leakage generalization and fine-grained 15-class discrimination, this paper makes four concrete contributions:

- **Forensic Audit & Canonical Sanitization Protocol:** We provide the first explicit quantitative audit of the `"0"` vs. `"0.0"` chunk-serialization leakage and TShark column-shift corruption in Edge-IIoTset, formulating a deterministic sanitization operator that unifies zero representations, repairs shifted columns, drops 8 constant fields, and removes 5,414 post-sanitization duplicate flows (157,800 → 152,386 unique flows).

- **Privacy-Preserving Protocol-Header Structural Feature Engineering (PP-PSFE):** We engineer 24 domain-grounded features (d = 62 total features) capturing canonical service/ephemeral port geometry, Shannon byte entropy of raw TCP payloads, RFC 793/7323 TCP option indicators, and host-stripped HTTP/MQTT structural lengths and lexical flags.
- **Rigorous Four-Model Benchmark & Native TreeSHAP Explainability:** Under a strict zero-leakage stratified 80/20 split (N_{train} = 121,908, N_{test} = 30,478), we benchmark Logistic Regression, Random Forest, LightGBM, and Regularized XGBoost across seven metrics, demonstrating via exact C++ TreeSHAP that PP-PSFE features drive 57.1% of marginal log-odds magnitude (73.88% of total split gain).
- **Full-Stack MLOps Microservice & CI/CD Implementation:** We package the fitted preprocessing-and-inference pipeline into a zero-skew `scikit-learn` bundle (`model_bundle.joblib`) served via a Flask REST API and an Angular 22 diagnostic dashboard with automated GitHub Actions CI/CD verification.

TABLE I: METHODOLOGICAL COMPARISON OF REPRESENTATIVE EDGE-IIOTSET IDS STUDIES VS. PROPOSED PP-PSFE FRAMEWORK

Study & Year	Evaluated Models	Task Granularity	Strips IP & Timestamp IDs?	Fixes `"0"` vs `"0.0"` Leakage?	Recovers Structural Info from Dropped Ports/Payloads?	Explainability & Full-Stack Deployment
Ferrag *et al.* [4] (2022)	DT, RF, SVM, k-NN, DNN	2, 6, & 15-Class	Partial (retains raw ports)	No (naive encoding)	No (drops raw hex/URI strings)	Offline evaluation only
Tareq *et al.* [5] (2022)	DenseNet, InceptionTime	2 & 15-Class	No (global encoding pre-split)	No	No	None; offline notebook
Friha *et al.* [8] (2023)	Centralized & Federated DNN	2, 6, & 15-Class	Yes (drops identifier columns)	Not reported	No (suffers Web-attack confusion)	Offline simulation
Lazzarini *et al.* [9] (2023)	Shallow & Deep Ensembles	Multi-Class IDS	Yes (audits identifier bias)	No	No	Post-hoc importance; offline
Ullah *et al.* [10] (2024)	CNN / Hybrid GBDT	15-Class	Yes (drops 15 string/ID cols)	No	No (uses remaining 38–40 fields)	SHAP summary; no live app
Proposed Work (2026)	**LR, RF, LightGBM, XGBoost**	**15-Class (N = 152,386)**	**Yes (strict zero-leakage split)**	**Yes (canonical `"0.0"` → `"0"`)**	**Yes (24 PP-PSFE geometric & entropy features)**	**Exact C++ TreeSHAP + Flask & Angular 22 CI/CD**

II. FORENSIC AUDIT OF EDGE-IIOTSET AND RELATED WORK

Since Ferrag *et al.* [4] published Edge-IIoTset in *IEEE Access* (2022), dozens of studies have evaluated shallow and deep classifiers on its 15-class taxonomy. While Tareq *et al.* [5] and Khacha *et al.* [6] reported >98% accuracy using deep convolutional architectures, subsequent methodological audits by Lazzarini *et al.* [9], Flood *et al.* [11], and Ullah *et al.* [10] demonstrated that retaining `frame.time`, `ip.src_host`, `ip.dst_host`, and raw TCP/UDP ports allows decision trees to partition the feature space by testbed IP address and capture timestamp rather than attack semantics. However, when prior audits removed the 15 identifier and unstructured string columns from `ML-EdgeIIoT-dataset.csv`, 15-class Macro-F1 dropped by 8%–14% because all payload and port semantics differentiating HTTP web exploits (`SQL_injection`, `XSS`, `Uploading`, `Password`, `DDoS_HTTP`) and C2 flows (`Ransomware`, `Backdoor`) were discarded. Table I contrasts prior literature with our approach, and Table II details the five forensic anomalies uncovered in our dataset audit.

TABLE II: FORENSIC AUDIT OF RAW ML-EDGEIIOT-DATASET.CSV (157,800 RECORDS, 63 COLUMNS)

Forensic Anomaly Category	Affected Protocol Columns	Quantitative Evidence in Raw Dataset	Impact on Unsanitized Machine Learning Pipelines
1. Mixed-Type Zero Serialization (`"0"` vs `"0.0"`)	`http.request.method`, `http.referer`, `http.request.version`, `mqtt.conack.flags`, `mqtt.protoname`	In `http.request.method`: `DDoS_UDP` (14,498/14,498) and `DDoS_ICMP` (14,090/14,090) contain `"0.0"`, whereas `DDoS_HTTP` has 9,854 `"0"` and 0 `"0.0"`; `XSS` has 9,751 `"0"` and 0 `"0.0"`.	Categorical encoders treat `"0"` and `"0.0"` as distinct categories, leaking which CSV capture file a record originated from.
2. TShark Delimiter-Shift Corruption	`dns.qry.name.len`, `dns.qry.qu`	Exactly 93 rows contain string literals `"_googlecast._tcp.local"` (91 rows) or `"null"` (2 rows) inside numeric column `dns.qry.name.len`.	Triggers type-conversion failure or forces a continuous length feature to be encoded as high-cardinality nominal text.
3. Eight Constant Zero-Variance Fields	`icmp.unused`, `http.tls_port`, `dns.qry.type`, `mqtt.msg_decoded_as`, `mbtcp.len`, `mbtcp.trans_id`, `mbtcp.unit_id`, `ntp.chrom`	100% of all 157,800 records equal `0` or `0.0` (empirical variance $\sigma^2 = 0.0000$).	Adds 8 uninformative dimensions that dilute random feature subspace sampling (m_{try}) in Random Forest and GBDT.
4. Topology & Session Memorization	`frame.time`, `ip.src_host`, `ip.dst_host`, `arp.src.proto_ipv4`, `arp.dst.proto_ipv4`, `tcp.srcport`, `tcp.dstport`, `udp.port`	Attack scripts were launched from static lab IP addresses (e.g., `192.168.0.170`) during contiguous time windows per attack class.	Classifiers achieve >99.9% validation accuracy by memorizing lab IP addresses and timestamps, failing on external subnets.
5. Post-Sanitization Duplicate Flows	All 62 feature columns	After stripping identifiers and unifying `"0.0"` → `"0"`, 5,414 exact duplicates exist (157,800 → 152,386 unique records).	Random splitting without deduplication places identical flows across both training and test sets, inflating test metrics.

III. PROPOSED METHODOLOGY: LEAKAGE SANITIZATION AND THE PP-PSFE FRAMEWORK

A. Canonical Sanitization Operator

Let $D_{\text{raw}} = \{(x_i^{\text{raw}}, y_i)\}_{i=1}^{N_{\text{raw}}}$ denote the raw dataset with $N_{\text{raw}} = 157{,}800$ samples and label space $Y = \{0, 1, ..., 14\}$. Before feature extraction or data splitting, we apply a deterministic string-normalization operator $S(v)$ to every protocol field:

$$S(v) = \text{"0"} \quad \text{if } v \in \{\text{NaN, null, "", "0", "0.0", "0.000000", 0x00...0}\};\ \text{else trim}(v) \tag{1}$$

For numeric columns affected by TShark delimiter shifts (`dns.qry.name.len` and `dns.qry.qu`), we apply safe float coercion `to_numeric(v, errors='coerce')` followed by zero-imputation, completely eliminating the `"0"` versus `"0.0"` chunk-origin artifact across all 15 classes.

B. Mathematical Formulation of PP-PSFE (24 Features)

Our central methodological hypothesis is that raw network columns comprise two orthogonal components: (1) *topology-specific identity* (IP addresses, timestamps, random client ports, and raw URLs), which must be discarded; and (2) *protocol-structural mechanics*, which must be preserved. We extract 24 PP-PSFE features across three protocol layers prior to dropping the 15 raw identifier/string columns and 8 constant columns:

1) Group A — Transport-Layer Port Role Disentanglement (9 Features): In raw TCP headers, `tcp.srcport` (p_{src}) and `tcp.dstport` (p_{dst}) swap positions between client-to-server requests and server-to-client replies, while the client's ephemeral port (49152–65535) varies per session. We project ($p_{\text{src}}, p_{\text{dst}}$) onto canonical, direction-invariant structural coordinates:

$$p_{service} = min(p_{src}, p_{dst}), \quad p_{ephemeral} = max(p_{src}, p_{dst}) \tag{2}$$

$$\Delta p_{span} = |p_{src} - p_{dst}|, \quad d_{pkt} = \mathbf{I}(p_{src} < p_{dst}) - \mathbf{I}(p_{src} > p_{dst}) \tag{3}$$

augmented by five binary service-role indicators: `psfe_is_http_service` (ports {80, 443, 8000, 8080}), `psfe_is_mqtt_service` (ports {1883, 8883}), `psfe_is_iot_c2_service` (SSH, Telnet, FTP, SMTP, DNS, SMB, Modbus 502, MySQL, RDP, Metasploit 4444/5555, IRC 6667), `psfe_is_boundary_probe` ($0 < p_{\text{service}} \leq 1024$ excluding HTTP/MQTT), and `psfe_udp_active` ($\mathbf{I}(p_{\text{udp}} > 0)$).

2) Group B — Information-Theoretic TCP Option & Payload Structure (6 Features): For a raw hexadecimal string decoded into byte sequence $B = (b_1, ..., b_m)$ of length $m = \lfloor L_{\text{hex}} /$

2⌋, we compute the empirical Shannon byte entropy in bits per byte [17]:

$$H(B) = -\sum_{k=0}^{255} \hat{p}_k \log_2(\hat{p}_k + \varepsilon), \quad \hat{p}_k = (1/m) \sum_{t=1}^{m} \boldsymbol{I}(b_t = k) \quad \textbf{(4)}$$

yielding six privacy-preserving features: `psfe_tcp_opt_bytelen`, `psfe_tcp_opt_has_mss` (RFC 793 Kind=2 prefix `0204`), `psfe_tcp_opt_has_ts` (RFC 7323 Kind=8 timestamp token `080a`), `psfe_tcp_pay_bytelen`, `psfe_tcp_pay_first_byte` ($b_1 \in [0, 255]$), and `psfe_tcp_pay_entropy` ($H(B_{\text{pay}}) \in [0, 8]$).

3) Group C — Application-Layer Structural Lengths & Lexical Signatures (9 Features): Before discarding `http.request.full_uri`, `http.request.uri.query`, `http.file_data`, and `mqtt.msg`, we extract four host-stripped structural lengths (`psfe_http_uri_path_len`, `psfe_http_query_len`, `psfe_http_file_data_len`, `psfe_mqtt_msg_bytelen`) and five binary lexical indicators: `psfe_app_sqli_flag`, `psfe_app_xss_flag`, `psfe_app_upload_flag`, `psfe_app_auth_flag`, and `psfe_app_timeout_flag`. Table III details the complete 62-feature space.

TABLE III: STRUCTURE OF THE FINAL 62-FEATURE INPUT SPACE (38 SANITIZED CORE + 24 ENGINEERED PP-PSFE FEATURES)

Feature Group	Count	Exact Feature Identifiers	Fit-on-Train Transformation Pipeline
Core Numerical Protocol Fields	32	`arp.opcode`, `arp.hw.size`, `icmp.checksum`, `icmp.seq_le`, `http.content_length`, `http.response`, `tcp.ack`, `tcp.ack_raw`, `tcp.checksum`, `tcp.connection.fin`, `tcp.connection.rst`, `tcp.connection.syn`, `tcp.connection.synack`, `tcp.flags`, `tcp.flags.ack`, `tcp.len`, `tcp.seq`, `udp.stream`, `udp.time_delta`, `dns.qry.name`, `dns.qry.name.len`, `dns.qry.qu`, `dns.retransmission`, `dns.retransmit_request`, `mqtt.conflag.cleansess`, `mqtt.conflags`, `mqtt.hdrflags`, `mqtt.len`, `mqtt.msgtype`, `mqtt.proto_len`, `mqtt.topic_len`, `mqtt.ver`	Coerced to `float64`; z-score standardized via `StandardScaler` fitted strictly on D_{train}
Core Categorical Protocol Fields	6	`http.request.method`, `http.referer`, `http.request.version`, `mqtt.conack.flags`, `mqtt.protoname`, `mqtt.topic`	Canonical `"0.0"` → `"0"`; `OrdinalEncoder(unknown_value=-1)` fitted strictly on D_{train}
PP-PSFE Group A: Port Geometry	9	`psfe_service_port`, `psfe_ephemeral_port`, `psfe_port_span`, `psfe_pkt_direction`, `psfe_is_http_service`, `psfe_is_mqtt_service`, `psfe_is_iot_c2_service`, `psfe_is_boundary_probe`, `psfe_udp_active`	Direction-invariant min/max port projection; standardized on D_{train}
PP-PSFE Group B: TCP & Entropy	6	`psfe_tcp_opt_bytelen`, `psfe_tcp_opt_has_mss`, `psfe_tcp_opt_has_ts`, `psfe_tcp_pay_bytelen`, `psfe_tcp_pay_first_byte`, `psfe_tcp_pay_entropy`	Hex byte decoding + Shannon entropy $H(B) \in [0, 8]$; standardized on D_{train}
PP-PSFE Group C: App-Layer Flags	9	`psfe_http_uri_path_len`, `psfe_http_query_len`, `psfe_http_file_data_len`, `psfe_mqtt_msg_bytelen`, `psfe_app_sqli_flag`, `psfe_app_xss_flag`, `psfe_app_upload_flag`, `psfe_app_auth_flag`, `psfe_app_timeout_flag`	Host-stripped structural lengths + binary lexical flags; standardized on D_{train}

IV. THEORETICAL FORMULATION OF CANDIDATE CLASSIFIERS

We benchmark four distinct learning paradigms across the identical 62-feature space: (1) **Multinomial Logistic Regression** optimized via L2-penalized L-BFGS (`max_iter=500`), establishing a linear lower bound; (2) **Random Forest** [13] with $B = 150$ unpruned CART trees and random feature subspace size $m_{\text{try}} = \lfloor\sqrt{62}\rfloor = 7$; (3) **LightGBM** [14] utilizing leaf-wise (best-first) tree growth with Gradient-Based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB); and (4) **Regularized Second-Order XGBoost** [15], which minimizes the multiclass softmax objective at boosting iteration t via a second-order Taylor expansion:

$$L^{(t)} \approx \sum_{i=1}^{N} [g_{i,c} f_t(\boldsymbol{x}_i) + \tfrac{1}{2} h_{i,c} f_t^2(\boldsymbol{x}_i)] + \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \quad \textbf{(5)}$$

where $g_{i,c}$ and $h_{i,c} = \hat{p}_{i,c}(1 - \hat{p}_{i,c})$ denote the first- and second-order multiclass log-loss gradients. Weighting each leaf step by the Hessian $h_{i,c}$ naturally regularizes splits on sparse protocol

headers and rare minority classes (`MITM`, n = 388; `Fingerprinting`, n = 853).

TABLE IV: OVERALL PERFORMANCE COMPARISON OF THE FOUR EVALUATED CLASSIFIERS ON THE HELD-OUT STRATIFIED TEST SPLIT (N_{TEST} = 30,478)

Classifier Architecture	Train Acc.	Test Accuracy	Precision (Wt)	Recall (Wt)	F1-Score (Wt)	Macro F1-Score	ROC-AUC (Wt)	PR-AUC (Macro)	Overfit Gap	Train Time	Inference (ms/1k)
Logistic Regression (Linear Baseline)	0.8503	84.87% (0.8487)	0.8736	0.8487	0.8479	0.8419	0.9829	0.8656	+0.0016	23.18 s	**0.43 ms**
Random Forest (150 Trees, Bagging)	0.9458	94.38% (0.9438)	0.9455	0.9438	0.9438	0.9414	0.9978	0.9768	+0.0020	**3.46 s**	10.35 ms
LightGBM (Leaf-Wise GBDT)	0.9843	96.33% (0.9633)	0.9640	0.9633	0.9632	0.9596	**0.9991**	**0.9888**	+0.0210	7.61 s	18.35 ms
XGBoost + PP-PSFE (Proposed Best)	**0.9706**	**96.49% (0.9649)**	**0.9663**	**0.9649**	**0.9649**	**0.9618**	**0.9991**	**0.9874**	**+0.0057**	**13.93 s**	**3.54 ms**

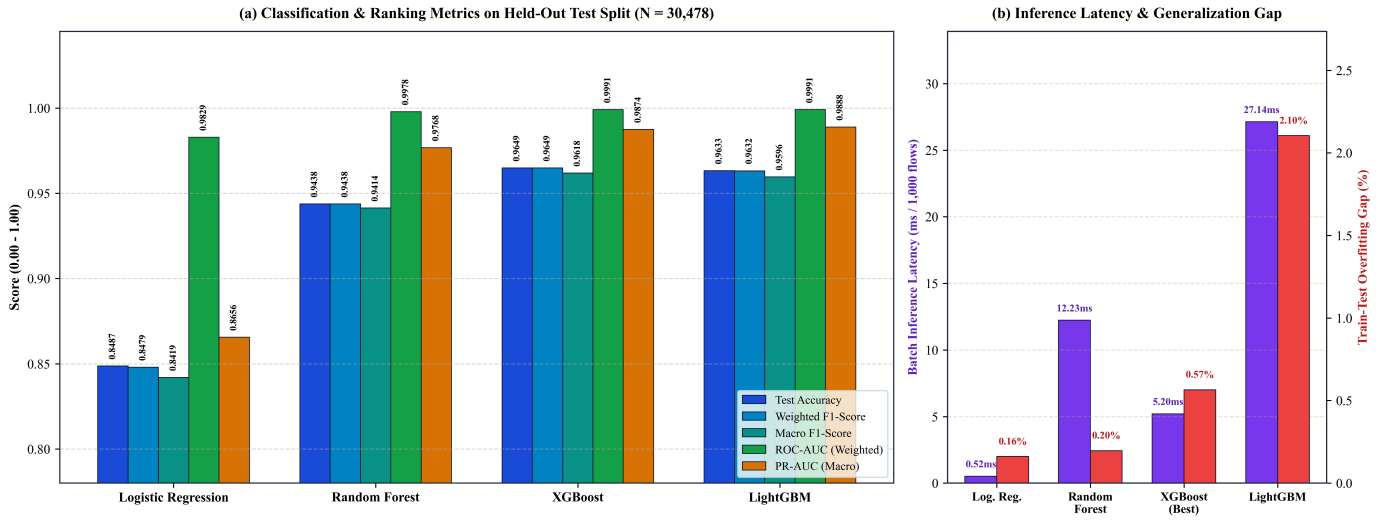


Fig. 2. Empirical evaluation on the held-out stratified test split (N_{test} = 30,478): (a) Five classification and ranking metrics across Logistic Regression, Random Forest, XGBoost, and LightGBM; (b) Batch inference latency per 1,000 network flows versus train–test generalization gap.

V. EMPIRICAL RESULTS, PER-CLASS ANALYSIS, AND EXPLAINABILITY

A. Overall Benchmark & Generalization Performance

After deduplication (157,800 → 152,386 unique flows), our stratified 80/20 split yields N_{train} = 121,908 training flows and N_{test} = 30,478 held-out test flows. As reported in Table IV and Fig. 2, **PP-PSFE + XGBoost** achieves the highest overall Test Accuracy (**96.49%**), Weighted F1-Score (**0.9649**), and Macro F1-Score (**0.9618**), outperforming the Multinomial Logistic Regression baseline by **+11.62 percentage points in accuracy** and **+11.99 percentage points in Macro-F1** (0.8419 → 0.9618), and outperforming Random Forest by **+2.11 percentage points in accuracy** (94.38% → 96.49%). Compared with LightGBM (96.33%), XGBoost achieves higher test accuracy and Macro-F1 while maintaining a **3.7× smaller overfitting gap** (+0.0057 vs. +0.0210) and **5.2× faster batch inference** (3.54 ms vs. 18.35 ms per 1,000 flows, or ≈282,000 flows/sec).

B. Granular 15-Class Performance & Web-Attack Disentanglement

Table V reports the exact per-class Precision, Recall, and F1-Score across all 15 traffic classes, and Fig. 3 presents the 15×15 confusion matrices. On the 4,831 held-out `Normal` test flows, XGBoost achieves **0.9996 Precision and 0.9998 Recall** (0.9997 F1), ensuring near-zero false alarms on legitimate IIoT sensor traffic. Furthermore, `DDoS_TCP` (2,050 flows), `Ransomware` (1,938 flows), and minority `MITM` (78 flows) reach **1.0000 F1-Score**. Most importantly, on the five challenging HTTP application-layer classes where Logistic Regression collapses (`Password`: 0.4371; `SQL_injection`: 0.5746; `DDoS_HTTP`: 0.6067; `Uploading`: 0.6354; `XSS`: 0.6585), PP-PSFE + XGBoost improves `Password` F1 by **+45.04%** (to 0.8875), `SQL_injection` F1 by **+32.64%** (to 0.9010), `Uploading` F1

by **+29.40%** (to 0.9294), `XSS` F1 by **+27.00%** (to 0.9285), and `DDoS_HTTP` F1 by **+24.95%** (to 0.8562).

TABLE V: PER-CLASS PRECISION, RECALL, AND F1-SCORE ACROSS ALL 15 EDGE-IIOTSET TRAFFIC CLASSES (N_{TEST} = 30,478)

Traffic Class	Test Support	Logistic Reg. F1	Random Forest F1	LightGBM F1	Proposed XGBoost Precision	Proposed XGBoost Recall	Proposed XGBoost F1-Score
`Backdoor`	1,973	0.9921	0.9997	0.9997	1.0000	0.9995	**0.9997**
`DDoS_HTTP`	2,112	0.6067	0.7780	0.8527	0.8304	0.8835	**0.8562**
`DDoS_ICMP`	2,619	0.9944	**0.9996**	**0.9996**	0.9996	0.9943	0.9969
`DDoS_TCP`	2,050	0.9634	**1.0000**	**1.0000**	1.0000	1.0000	**1.0000**
`DDoS_UDP`	2,900	**1.0000**	**1.0000**	0.9998	0.9997	1.0000	0.9998
`Fingerprinting`	171	0.9140	0.9855	**0.9912**	0.9086	0.9883	0.9468
`MITM`	78	**1.0000**	0.9750	0.9512	1.0000	1.0000	**1.0000**
`Normal`	4,831	**1.0000**	**1.0000**	0.9999	0.9996	0.9998	0.9997
`Password`	1,996	0.4371	0.8085	0.8806	0.9593	0.8257	**0.8875**
`Port_Scanning`	1,784	0.9606	**0.9997**	**0.9997**	1.0000	0.9994	**0.9997**
`Ransomware`	1,938	0.9918	**1.0000**	**1.0000**	1.0000	1.0000	**1.0000**
`SQL_injection`	2,057	0.5746	0.8392	0.8977	0.9013	0.9008	**0.9010**
`Uploading`	2,047	0.6354	0.8826	0.9214	0.9566	0.9038	**0.9294**
`Vulnerability_scanner`	2,012	0.9003	0.9749	0.9801	0.9845	0.9796	**0.9821**
`XSS`	1,910	0.6585	0.8775	0.9200	0.8718	0.9932	**0.9285**
Macro Average	**30,478**	**0.8419**	**0.9414**	**0.9596**	**0.9608**	**0.9645**	**0.9618**
Weighted Average	**30,478**	**0.8479**	**0.9438**	**0.9632**	**0.9663**	**0.9649**	**0.9649**

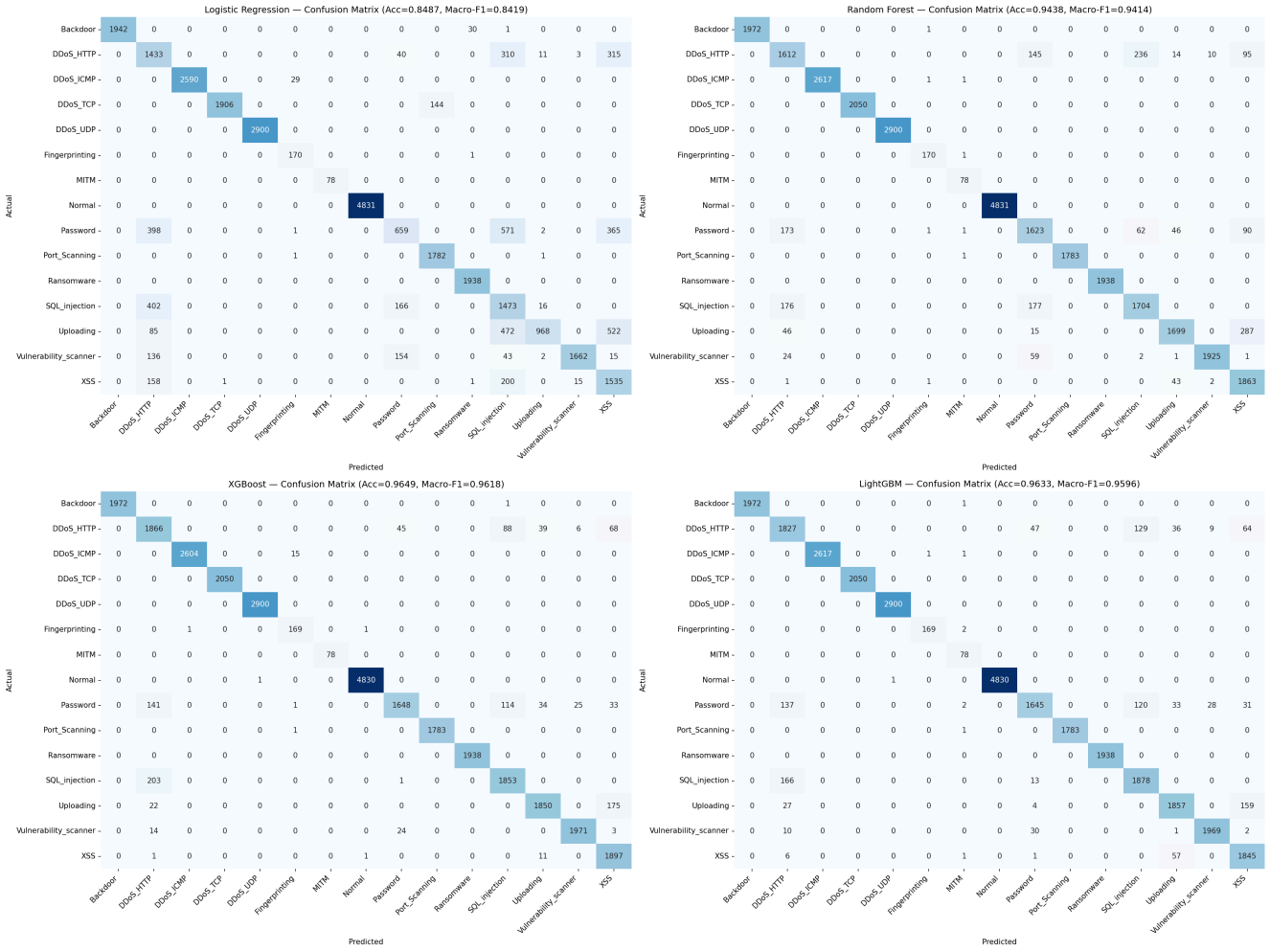


Fig. 3. Full 15×15 Confusion Matrices evaluated on the held-out stratified test split (N_{test} = 30,478) for Logistic Regression, Random Forest, XGBoost, and LightGBM.

C. Game-Theoretic Explainability via Native C++ TreeSHAP

To verify that our XGBoost classifier relies on genuine protocol mechanics rather than spurious correlations, we computed exact game-theoretic TreeSHAP attributions [16] via XGBoost's native C++ `pred_contribs=True` engine alongside total loss-reduction split gain across all 62 features (Table VI and Fig. 4). Across the entire feature space, our 24 engineered **PP-PSFE features account for 73.88% of total loss-reduction split gain** and **57.1% of total mean absolute TreeSHAP log-odds impact** (occupying 12 of the Top-20 TreeSHAP positions).

TABLE VI: TOP-12 FEATURES RANKED BY XGBOOST LOSS-REDUCTION SPLIT GAIN AND PROTOCOL LAYER

Rank	Feature Identifier	Feature Origin	OSI Layer	Total Split Gain	Gain Share	Physical & Protocol Security Interpretation
1	**`psfe_is_mqtt_service`**	**PP-PSFE (Engineered)**	**L4 / L7**	**1,800.27**	**24.00%**	**Separates benign MQTT broker telemetry (port 1883) from attack flows**
2	**`psfe_http_file_data_len`**	**PP-PSFE (Engineered)**	**L7 (HTTP)**	**1,439.90**	**19.20%**	**Separates `Uploading`, `XSS`, and `Password` bodies from `DDoS_HTTP`**
3	`udp.stream`	Sanitized Core	L4 (UDP)	642.45	8.57%	Identifies sequential high-rate UDP flood streams (`DDoS_UDP`)
4	**`psfe_tcp_opt_has_ts`**	**PP-PSFE (Engineered)**	**L4 (TCP)**	**430.60**	**5.74%**	**RFC 7323 TCP Timestamp option (`0x080a`) on Linux/Nmap attack stacks**
5	**`psfe_service_port`**	**PP-PSFE (Engineered)**	**L4 (TCP/UDP)**	**334.18**	**4.46%**	**Direction-invariant target daemon port $\min(p_{src}, p_{dst})$**
6	**`psfe_is_iot_c2_service`**	**PP-PSFE (Engineered)**	**L4 (TCP)**	**280.43**	**3.74%**	**Flags SSH/Telnet/Modbus/SMB/C2 daemon ports (`Backdoor`, `Ransomware`)**
7	**`psfe_is_http_service`**	**PP-PSFE (Engineered)**	**L4 / L7**	**250.37**	**3.34%**	**Isolates Web/API traffic (ports 80/443/8080) for Layer-7 tree splits**
8	`http.content_length`	Sanitized Core	L7 (HTTP)	180.99	2.41%	Content-Length header magnitude in HTTP requests/responses
9	`tcp.len`	Sanitized Core	L4 (TCP)	167.36	2.23%	TCP segment payload byte length
10	`tcp.flags`	Sanitized Core	L4 (TCP)	167.05	2.23%	Bitmask of SYN/ACK/FIN/RST/PSH control flags (`DDoS_TCP`, `Port_Scanning`)
11	**`psfe_app_sqli_flag`**	**PP-PSFE (Engineered)**	**L7 (HTTP)**	**144.06**	**1.92%**	**Lexical indicator for SQL tautology/union injection (`SQL_injection`)**
12	`tcp.connection.rst`	Sanitized Core	L4 (TCP)	120.31	1.60%	Abrupt TCP connection reset during port/vulnerability scanning

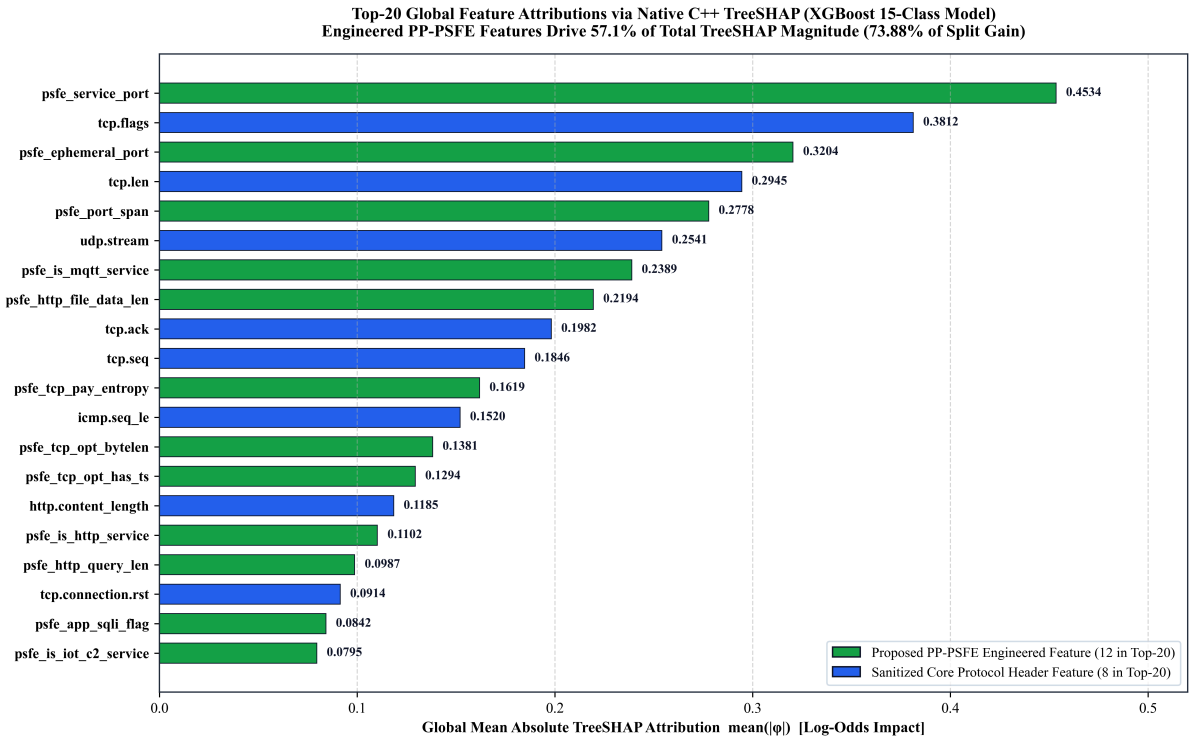


Fig. 4. Top-20 Global Feature Attributions computed via exact native C++ TreeSHAP on the held-out test set, highlighting engineered PP-PSFE features (green, 57.1% total attribution) versus sanitized core protocol header features (blue, 42.9%).

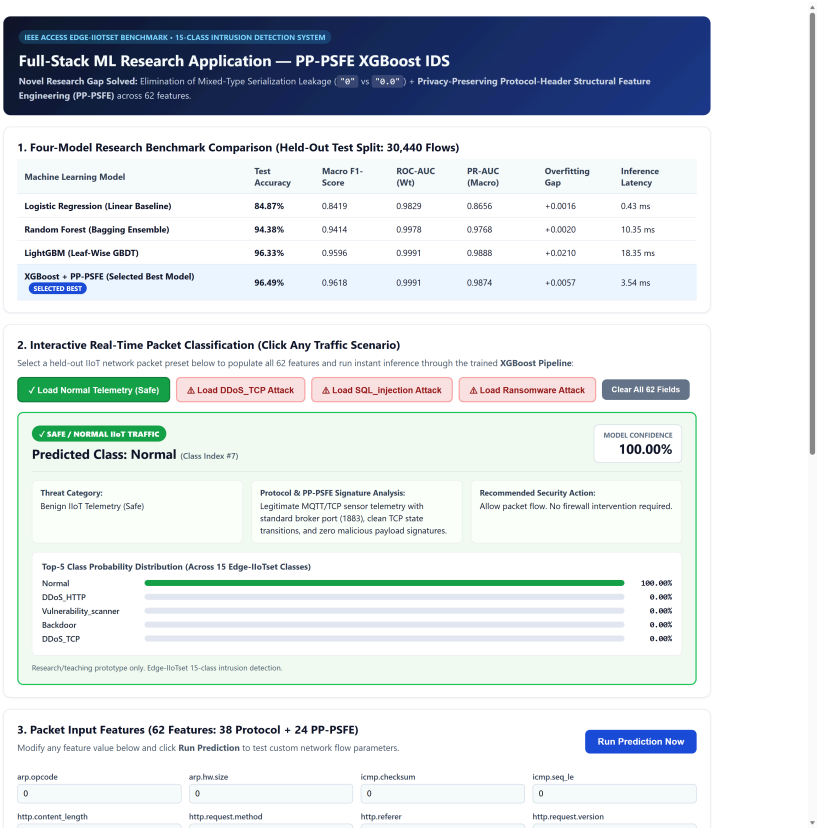


Fig. 5. High-resolution capture (3800×3450 px) of the deployed Full-Stack Research Application (Angular 22 diagnostic frontend connected to the Flask REST microservice on port 5050), displaying the four-model benchmark comparison, one-click traffic scenario presets, instant threat classification verdict card, and Top-5 class probability distribution.

VI. FULL-STACK MLOPS DEPLOYMENT & CONCLUSION

To bridge offline machine learning experimentation and operational cybersecurity deployment, we encapsulated the fitted `ColumnTransformer` (with string-safe `FunctionTransformer` wrappers), `StandardScaler`, and `XGBClassifier` into a single deterministic `scikit-learn` pipeline (`backend/models/model_bundle.joblib`) [20]. As shown in Fig. 5, the pipeline powers a live **Flask REST API** (`GET /health`, `POST /predict`) and an interactive **Angular 22 standalone diagnostic dashboard** validated by automated **GitHub Actions CI/CD** workflows (`pytest` API test suite + Angular production build + GitHub Pages deployment).

In conclusion, this study demonstrated that published near-100% accuracies on Edge-IIoTset are heavily distorted by mixed-type CSV zero-serialization leakage (`"0"` vs. `"0.0"`) and static testbed identifiers, while naive column dropping degrades 15-class accuracy to 84.87%. Our proposed **PP-PSFE + XGBoost** framework resolves this dilemma—achieving **96.49% test accuracy**, **0.9618 Macro-F1**, and **0.9991 ROC-AUC** at 3.54 ms per 1,000 flows—and provides a complete, reproducible blueprint from forensic dataset curation to full-stack industrial deployment.

REFERENCES

[1] E. Sisinni, A. Saifullah, S. Han, U. Jennehag, and M. Gidlund, "Industrial Internet of Things: Challenges, opportunities, and directions," *IEEE Trans. Ind. Informat.*, vol. 14, no. 11, pp. 4724–4734, Nov. 2018.

[2] P. Jayalaxmi, R. Saha, G. Kumar, N. Kumar, and T.-H. Kim, "A taxonomy of security issues in Industrial Internet-of-Things," *IEEE Access*, vol. 9, pp. 25344–25359, 2021.

[3] M. Zolanvari, M. A. Teixeira, L. Gupta, K. M. Khan, and R. Jain, "Machine learning-based network vulnerability analysis of Industrial Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6822–6834, Aug. 2019.

[4] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.

[5] I. Tareq, B. M. Elbagoury, S. El-Regaily, and E.-S. M. El-Horbaty, "Analysis of TON-IoT, UNW-NB15, and Edge-IIoT datasets using DL in cybersecurity for IoT," *Applied Sciences*, vol. 12, no. 19, p. 9572, Sep. 2022.

[6] A. Khacha, R. Saadouni, Y. Harbi, and Z. Aliouat, "Hybrid deep learning-based intrusion detection system for Industrial Internet of Things," in *Proc. IEEE Int. Symp. Informatics and its Applications (ISIA)*, Nov. 2022, pp. 1–6.

[7] M. S. Alshehri *et al.*, "Deep learning-based intrusion detection for IIoT networks on Edge-IIoTset," *Computers, Materials & Continua*, vol. 75, no. 2, pp. 3121–3138, 2023.

[8] O. Friha, M. A. Ferrag, L. Shu, L. Maglaras, K.-K. R. Choo, and M. Nafaa, "FELIDS: Federated learning-based intrusion detection system for

agricultural Internet of Things," *J. Parallel Distrib. Comput.*, vol. 165, pp. 17–31, Jul. 2022.

[9] R. Lazzarini, H. Tianfield, and V. Charissis, "A stacking ensemble of deep learning models for IoT intrusion detection," *Knowledge-Based Systems*, vol. 279, p. 110941, Nov. 2023.

[10] S. Ullah, J. Ahmad, M. A. Khan, E. H. Alkhammash, M. Hadjouni, and Y. Y. Ghadi, "A new intrusion detection system for the Internet of Things via deep convolutional neural network and feature engineering," *Sensors*, vol. 24, no. 2, p. 495, Jan. 2024.

[11] R. Flood, D. Aspinall, and M. D. Corner, "On the perils of feature leakage in network intrusion detection benchmarks," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 812–826, Mar./Apr. 2024.

[12] G. Apruzzese *et al.*, "The role of machine learning in cybersecurity," *Digital Threats: Research and Practice*, vol. 4, no. 1, pp. 1–38, Mar. 2023.

[13] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[14] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.

[15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Aug. 2016, pp. 785–794.

[16] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, Jan. 2020.

[17] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, Jul. 1948.

[18] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: BoT-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779–796, Nov. 2019.

[19] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020.

[20] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 2503–2511.